Debugging strategies

Module 5 - Lesson 2

Overview

Bugs are inevitable; debugging is a skill. Reading tracebacks, narrowing the problem, and using the right tools fix most issues quickly.

Key Concepts

- Read the traceback bottom-up: the last line names the exception and location.
- Reproduce the bug with the smallest possible input before fixing.
- Print or log intermediate values to verify assumptions; better, use pdb.
- pdb (breakpoint()) pauses execution and lets you inspect state interactively.
- Split a failing expression into steps to isolate which part breaks.
- Write a failing test for the bug, then fix until the test passes.

Example

```
def process(items):
    breakpoint()   # pause here to inspect items
    return [i * 2 for i in items]

# pdb commands:
# (Pdb) items
# (Pdb) n   (next)
# (Pdb) q   (quit)
```

Summary

Debugging is systematic: read the traceback, minimize the repro, and inspect state. breakpoint() and failing tests turn mystery crashes into quick fixes.

Practice Checklist

- Introduce a bug, read the traceback, and fix it from the message alone.
- Use breakpoint() to inspect variables mid-function.
- Write a regression test for a bug you fixed.